Critical Infrastructure Security

IY5612 (IY3612/IY4612) — Academic Year 2025/2026 (Unit 5)

Stephen D. Wolthusen
stephen.wolthusen@rhul.ac.uk

Department of Information Security and Communication Technology
Norwegian University of Science and Technology, Norway

and

Department of Information Security
Royal Holloway, University of London, United Kingdom

Unit 5— Critical Infrastructures Interdependency Models

Contents

CI Interdependence Models

CI Interdependency Models
- Introduction
- Graph-Based Models
- Agent-Based Models
- Game-Theoretical Models

Part I

CI Interdependence Models

CI Interdependency Models

This block describes some finer-grained dependency models used in understanding vulnerabilities and adversarial behaviour

Such models can be used at different abstraction levels ranging from entire critical infrastructure sectors to structures within an organisation that may be susceptible to attacks by sophisticated adversaries

Models covered include:

- Graph-based models
- Agent-based approaches
- Game-theoretical models

Recommended Reading

- N. K. Svendsen, S. Wolthusen:
  Modelling Approaches
  In: J. Lopez, R. Setola, S. Wolthusen (eds.):
  Critical Infrastructure Protection: Information Infrastructure Models, Analysis, and Defense
  Lecture Notes in Computer Science Vol. 7130, Springer-Verlag, 2012

Graph-Based Models

As shown throughout this module, graphs provide a flexible and elegant way of capturing both networks and more generally relationships among entities

- In the following we are particularly interested in (inter-)dependencies
- This naturally leads us to consider directed graphs as noted previously

In the simplest case, relations expressed as **dependency graphs** are well-understood and also found in many areas of computer science and mathematics

- This can be further embellished by assigning weights or other properties to edges, assigning further attributes to nodes, or both
- Attributes will aggregate properties to some extent, and may provide abstraction of lower-layer models

Basic Model Properties (1)

When seeking to enhance the robustness of a network, minimising possible chains of dependencies is highly desirable, this can be expressed as the path length $l(u, v)$

- Recall that this gives a walk with distinct vertices – we may also consider trails (i.e. with distinct edges)

We can therefore (for a distance d) seek to minimise individual paths or the graph diameter

$$diam(G) = \max\{d(u, v) | u, v \in V(G)\}$$

or the average path length $\bar{d}(G)$

$$\bar{d}(G) = \frac{1}{|V|} \sum_{u \in V} \bar{d}(u) = \frac{1}{|V|^2 - |V|} \sum_{u,v \in V, u \neq v} d(u, v)$$

as well as the mean path length

Basic Properties (2)

A number of other basic graph properties gives us useful information:

- Is the graph strongly connected, k-connected? This gives a basic understanding of how many simultaneous (targeted) failures we can tolerate before connectivity is lost — for random edge removal, different models are more suitable
  - More general questions previously touched upon hinge on the degree distribution of a graph
- Conversely, we may be interested in identifying cut edges for graphs that are not k-connected

Centrality

Centrality and **Betweenness** (or more formally *Closeness Centrality* and *Betweenness Centrality* are important characteristics for understanding the vulnerability of network elements

- Recall that centrality intuitively gives a metric for the proximity of a node to all other nodes:
  - A node with high centrality has short paths to other nodes in the network
- Betweenness gives the total number of possible paths that pass through a node:
  - This indicates the fraction of paths from some node *i* to another node *k* passing through node *j*
  - Mainly used in the study of social networks, this can also give important insights in arbitrary other networks
  - Even for networks that are carefully designed, these properties can be surprising and are not at all obvious e.g. from placement or network traffic

Centrality Metrics
Centrality and Betweenness Centrality

Computing characteristics such as centrality and betweenness is computationally expensive, often requiring approximations for larger ($\approx 10^3$ node) networks

- Determining betweenness requires computing shortet paths between any two nodes, requiring e.g. $\mathcal{O}(n^3)$ complexity using the Floyd-Warshall algorithm
- However, it is nevertheless interesting to also study the dynamic behaviour of networks

Consider the case of modelling a sequence of targeted attacks on a network:

- Identifying nodes with the highest betweenness identifies likely targets is an obvious question
- However, an attacker may "pick off" nodes: *How is betweenness distributed for the remaining network?*

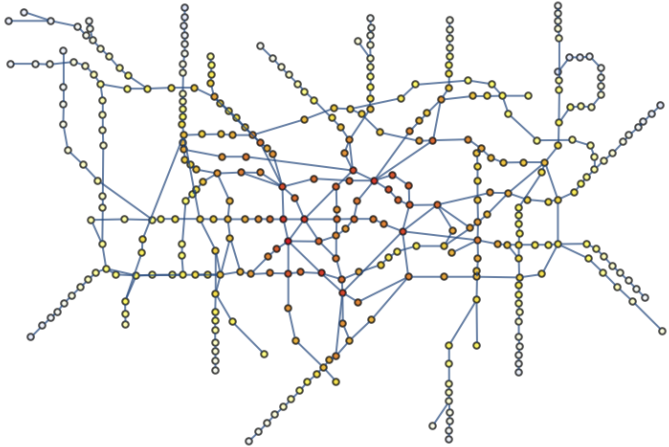
Centrality Examples

Consider the following graph of a reasonably well-known network:

(Source: Wolfram Research)

Example: Closeness Centrality

The closeness centrality C_C of a node u is defined as $\sum_{v \in V(G)} \delta(u, v)$ for a distance δ, identified here by darker colours:



Example: Closeness Centrality

Now consider betweenness centrality, here highlighting the ten stations in the Tube network that are most-traversed along shortest paths:

Clustering Coefficients

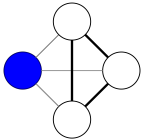
Also as noted before, Clustering Coefficients can be used to study the robustness of networks:

- These coefficients can be seen as intuitively showing how closely a graph is connected
- This can be defined globally based on the fraction of closed triangles over the total number of triplets
- Local clustering as introduced by Watts and Strogatz identifies how close neighbours are to forming a complete graph (clique)
- The network average clustering coefficient can then be definied as the average of the local clustering coefficients for all vertices

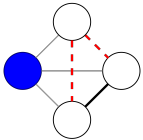
Local Clustering Coefficients

The local clustering coefficient is the fraction of edges between immediately connected neighbouring vertices divided by the number of possible edges

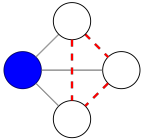
- Note that this is distinct for directed and undirected graphs
- In this example we only consider the undirected case
- Thick lines indicate realised edges for the neighbourhood of the blue node, red dashed lines possible but un-realised edges



c = 1



c = 1/3



Robustness in Scale-Free Networks (1)

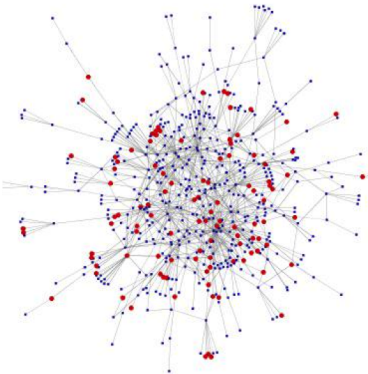
The degree distribution in a scale-free graph is skewed with most vertices having only a relatively small degree

- We have previously reviewed the general properties of such networks and considered one example in the case of Internet robustness
- The degree distribution has important implications:

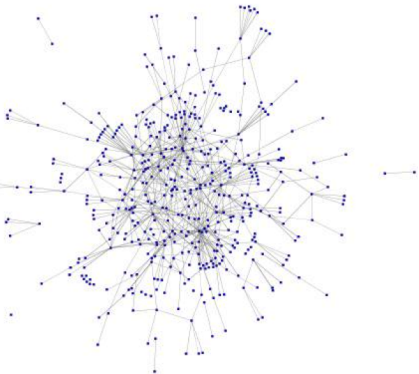
Average (local) clustering coefficients and centrality metrics will be relatively low

- Consider the following small example of a peer-to-peer network (for the historic Gnutella file sharing network)

Robustness in Scale-Free Networks (2)

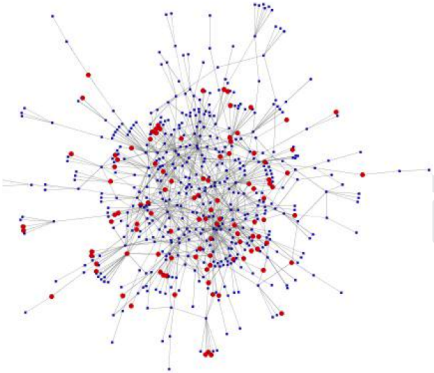


Basic network: 574 nodes in giant component



20% of nodes randomly removed: 427 nodes remaining in giant (connected) component

Robustness in Scale-Free Networks (3)



Basic network: 574 nodes in giant component

20% of nodes with highest clustering coefficient removed: 301 nodes remaining in giant (connected) component

Network Robustness
Flow Problems (1/3)

Most modelling described so far has considered only connectivity — as described before, this is not always adequate to capture desired properties:

- Most network links (edges) have finite **capacity**, and for nodes we may wish to consider its **load** as an additional element in the model
- This refined abstraction captures a number of networks, ranging from computer networks to electric power grids
  - Not every network is captured: Edges are assumed to be independent from one another — this is not the case for radio frequency networks

This allows the study of **flow problems** in networks to study their robustness

- This is a well-researched area studying mainly the capacity of networks: Typical problems are determining the maximum flow capacity between one (or all) sources and one (or several) sinks

Network Robustness
Flow Problems (2/3)

One formulation of the maximum flow problem arose from a classified study of the Soviet rail transportation network:

> *"Consider a rail network connecting two cities by way of a number of intermediate cities, where each link of the network has a number assigned to it representing its capacity. Assuming a steady-state condition, find a maximal flow from one given city to the other"* *(Harris, 1955)*

For flow problems, we will generally want to consider directed graphs $G = (V, E)$ with each edge $(u, v) \in E$ having a **capacity** $c(u, v) \geq 0$ (if $(u, v) \notin E$), then $c(u, v) = 0$), usually with designated source nodes s and t

Network Robustness
Flow Problems (3/3)

For a flow f to be considered legitimate, capacity constraints must be considered ($\forall (u,v) \in E : f(u,v) \leq c(u,v)$), and flow conservation is required:

$$\forall u \in V : u \neq s \text{ and } u \neq t \implies \sum_{w \in V} f(u,w) = 0$$

Flows can now be summed up over all possible paths; any remaining unused capacity between s and t is called the residual network G_f for a given flow f

- One of the simplest formulations is for integral flows with one source and one sink
- The basic Ford-Fulkerson algorithm solves this with $\mathcal{O}(E \max |f|)$, more efficient algorithms exist, e.g. by King, Rao, and Tarjan ($\mathcal{O}(VE)$)

Maximum Flow

Example: Ford-Fulkerson Algorithm (1/2)

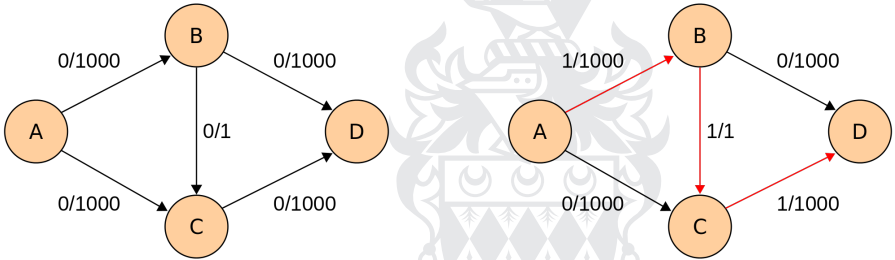
Input: Graph $G = (V, E)$ with flow capacity c, source s, and sink t.
Output: A flow f from s to t which is a maximum for all edges in E.

$f(u, v) \leftarrow 0$ for each edge $uv \in E$;

while there is an s-t path p in G_f such that $c_f(e) > 0$ for each edge $e \in E$ **do**
  find $c_f(p) = \min\{c_f(u, v) \mid (u, v) \in p\}$;
  for each edge $uv \in$ **do**
    $f(u, v) = f(u, v) + c_f(p)$;
    $f(v, u) = f(v, u) - c_f(p)$;

Maximum Flow

Example: Ford-Fulkerson Algorithm (2/2)



This example shows the first (of 2000!) steps in the worst-case behaviour of the Ford-Fulkerson algorithm — clearly, modern algorithms not exhibiting dependence on $\max|f|$ are desirable!

- Also, there are networks for which the Ford-Fulkerson algorithm will not terminate, so this should be considered only for simplicity

Cuts in Flow Problems

The principle underlying the design of the Ford-Fulkerson algorithm is to find the **minimum cut** remaining in the network

- By a cut (S, T) of G we mean a partition of V into S and $T = V \setminus S$ such that $s \in S$ and $t \in T$
- The flow across the cut (S, T) is then abbreviated as $f(S, T)$
- The capacity of the cut is similarly $c(S, T)$
- A **minimum cut** is then a cut with minimum capacity

By studying different criteria for cuts, we can now identify e.g. how to minimise the cost of disrupting a network — or conversely, to maximise the capacity loss

- This was the main interest in the previously mentioned study of the Soviet rail transportation network

Dependencies and Flows

If desired flows approach or exceed capacity, congestion or overloading is a typical consequence: The effects vary depending on the type of networks

- For most packet-based networks, congestion results in degraded quality of service and decreased overall throughput with rapid and severe degradation
- Similar phaenomena arise in computer networks as well as in e.g. traffic: Design objective for e.g. packet switched networks is not to exceed 80% of nominal capacity
- We note that more accurate models require to take buffers into account

Another flow problem that requires some additional consideration is the **electric grid**

- Physical laws determine power flows, although recent improvements in control have altered this somewhat

Cascading Failures

A particularly harmful failure mode is a **cascading** failure

- These can again occur in many networks, with computer networks and electric grids as the most prominent examples — but also very prominently in financial networks

Cascading Failure

Failures arise when one or more vertices or edges becomes unavailable (or its capacity is impaired), causing flows to be re-directed towards other nodes which in turn become overloaded

The search for alternative flow routes and inefficiencies can cause further disruption — this requires (and causes) further activity on the same networks

Understanding Cascading Failures

Modelling cascading failures in networks requires an understanding of relevant flows in the network and is necessarily dynamic

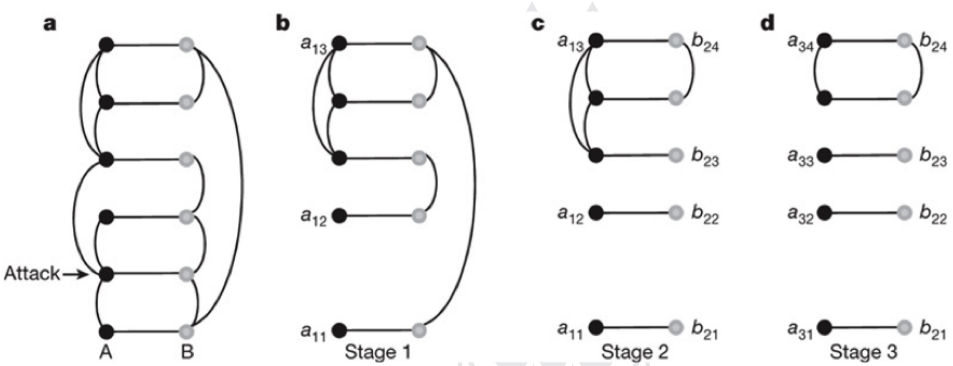
- Networks will usually not settle in an equilibrium state after a disturbance, but further disturbances (normal or malicious) will occur
- This requires capturing dynamic changes, behaviour will often be time-sensitive, e.g. where faults are cleared or repaired

Compensating for faults does not only require re-distributing the original load (flows), but often also requires less efficient work-arounds:

- Flows must be re-directed, reducing capacity
- This clearly suggests attack strategies minimising the cost of adversaries

Cascading Failure Example

Iterative Process as a Cascading Failure

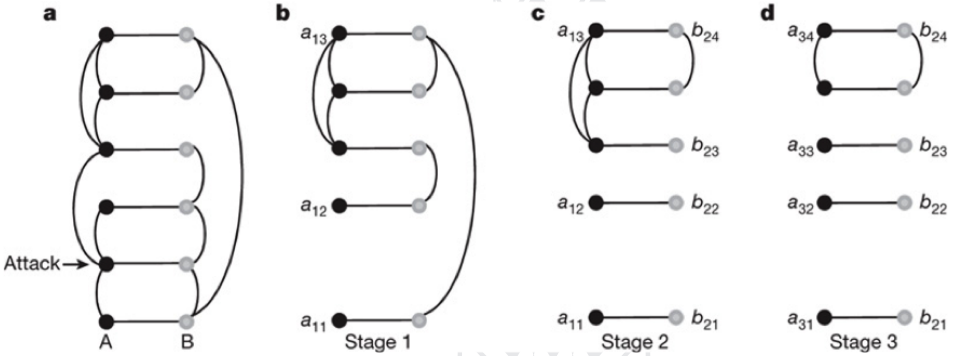


Each node in network *A* depends on only one node in network B and vice versa.
In (a), one node from *A* is removed

Source: S.V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin:
Catastrophic Cascade of Failures in Interdependent Networks Nature **464**:1025-1028 (Apr. 2010)

Cascading Failure Example

Iterative Process as a Cascading Failure

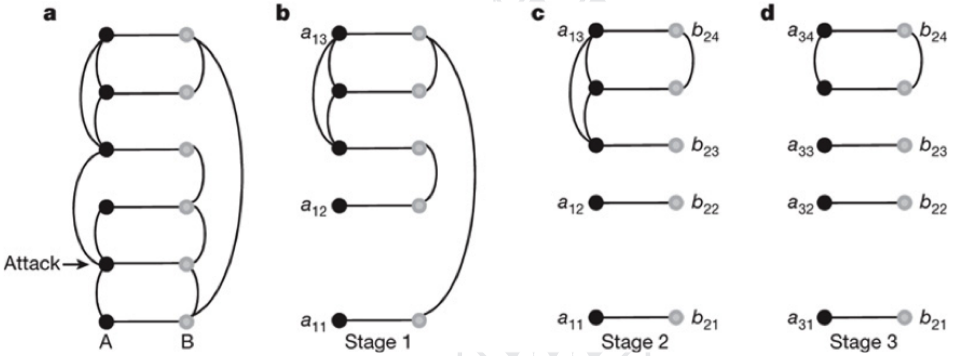


In (b), a dependent node in network *B* is also eliminated, resulting in network A breaking into three a_1 clusters a_{11}, a_{12}, and a_{13}

Source: S.V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin:
Catastrophic Cascade of Failures in Interdependent Networks Nature **464**:1025-1028 (Apr. 2010)

Cascading Failure Example

Iterative Process as a Cascading Failure

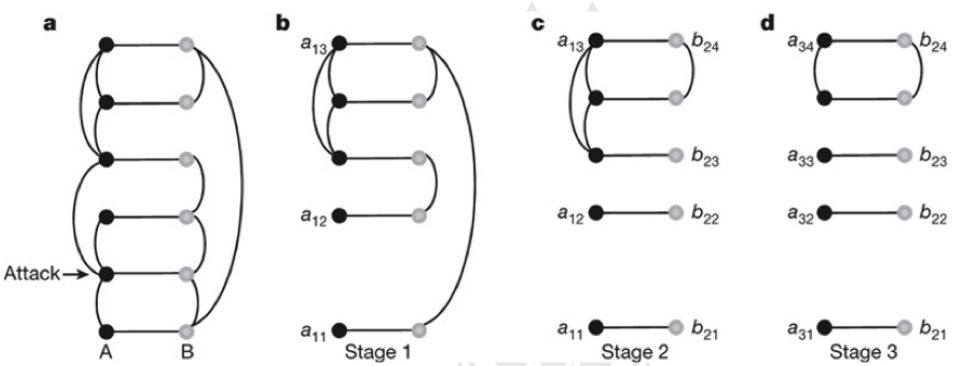


In (c) we remove links in *B* linking sets of nodes in *B* to separate clusters in a_1, resulting in *B* breaking into four b_2 clusters: b_{21}, b_{22}, b_{23}, and b_{24}

Source: S.V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin:
Catastrophic Cascade of Failures in Interdependent Networks Nature **464**:1025-1028 (Apr. 2010)

Cascading Failure Example

Iterative Process as a Cascading Failure



In (d), A-links connecting sets of A nodes to separate b_2 clusters are removed, resulting in A breaking into four a_3 clusters: These are identical with the b_2 clusters, so no further cascades occur

Source: S.V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin: *Catastrophic Cascade of Failures in Interdependent Networks* Nature **464**:1025-1028 (Apr. 2010)

Agent-Based Models (1/2)

Where interactions among network components beyond simple edge weights and particularly vertex labels are required, graphs models may be unsuitable

- Where state and possibly behaviour of vertices must be captured by a model, agent-based modelling can be helpful
- This behaviour can include physical processes, but may also aim to approximate social agents

As an example of the interaction between social networks and computer networks consider the effect of messages e.g. on Twitter/X

- In addition to re-transmission within the network, this can affect physical and computer networks: Overloading web servers (“*slashdotting*”)

Agent-Based Models (2/2)

Agent-based models require a precise description of the behaviour of each agent based on current state and external influences

- This may not be completely known even in completely benign cases, introducing uncertainty
- More importantly, if agents are controlled by an **adversary**, we may by definition not know the adversary's behaviour

Agent-based models are generally used extensively in simulations where an enhanced understanding of a network's behaviour under unusual circumstances is desired

- Wireless (including carrier) networks are a primary example, often also including agent mobility as an additional problem

Agent-Based Modelling Example

Simulating the Tor Overlay Network (1/2)

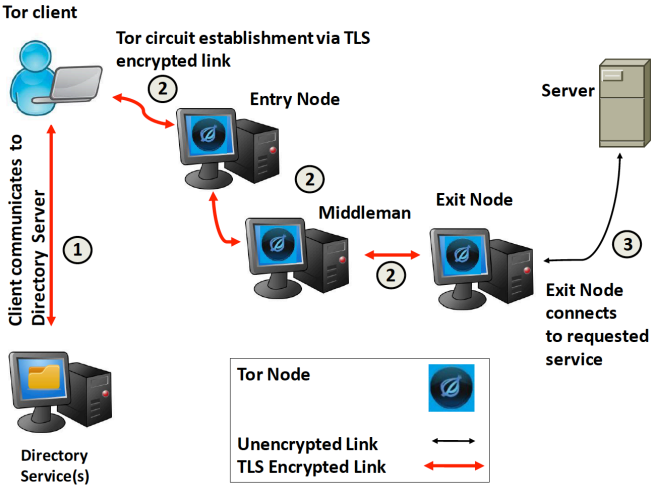
The Tor network is an overlay network aiming to provide privacy and anonymity for web traffic by

- Hiding which client connects to a given host
- Hiding the location of a service, or providing alternative access paths to blocked services

The service relies on a large, geographically distributed group of individuals “mixing” their network traffic, and thereby obfuscating it

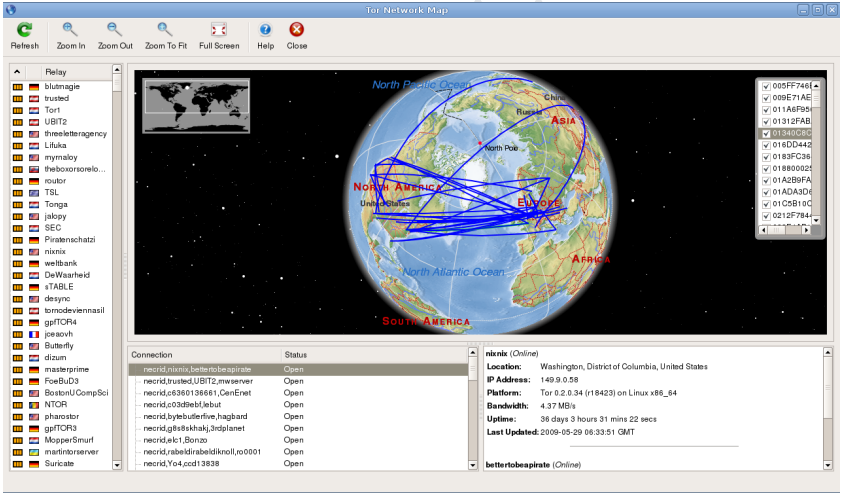
- If this network is infiltrated by sufficient adversaries, the mixing becomes imperfect and visible to the adversary
  - Adversary capabilities vary greatly, but may include nation states
- Maintaining anonymity can — for some users — be extremely critical
  - Political activists, but also criminals pursuing activities on the “dark net”

The Tor Overlay Network



High level flow of Tor communication: From entry node directory (1) to circuit using multiple intermediate nodes (2) to forwarding traffic through the newly established circuit (3)

The Tor Overlay Network



Source: The Tor project

Agent-Based Modelling Example

Simulating the Tor Overlay Network (2/2)

Tor operates on the principle of **onion routing**, with many layers of intervening nodes forwarding traffic with limited information on predecessors passed onwards

- Attackers will seek to either block access, or remove the anonymity provided by the onion routing

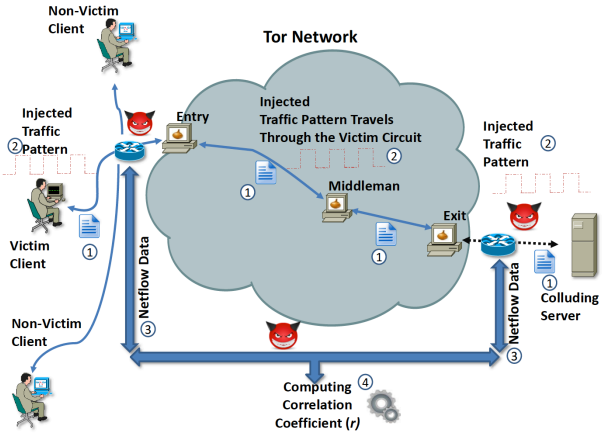
As Tor is an overlay network, modifications to the underlying network will affect is performance and also basic connectivity

- Adversaries can not only infiltrate the network, provide false entry *guards*, etc. but also influence clients in their ability and preference to pick entry guards

Key problem here: Changing guards too frequently increases the risk of denial of service, too rarely the risk of compromised guards

The Tor Overlay Network

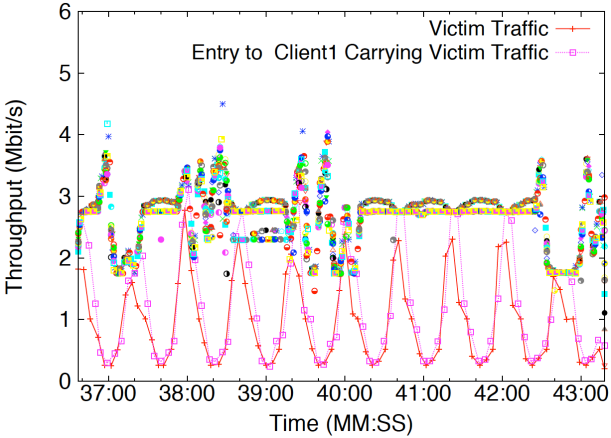
Unmasking Attacks



Using NetFlow traffic analysis to detect correlation between injected client and exit traffic for unmasking clients

Source: S. Chakravarty, M. V. Barbera, G. Portokalidis, M. Polychronakis and A. D. Keromytis: On the Effectiveness of Traffic Analysis Against Anonymity Networks Using Flow Records Proc. Passive and Active Network Management 2014, pp.247–257

The Tor Overlay Network
Unmasking Attacks



Visibility of injected traffic pattern

Source: S. Chakravarty, M. V. Barbera, G. Portokalidis, M. Polychronakis and A. D. Keromytis: On the Effectiveness of Traffic Analysis Against Anonymity Networks Using Flow Records Proc. Passive and Active Network Management 2014, pp.247–257

Game-Theoretical Models (1)

A related problem to describing the behavior of interacting entities is to capture interaction which is not simply captured by well-defined protocols

- This can be interaction with adversaries, but also between individual entities seeking to optimise their own benefit

As an example consider the use of congestion control in packet networks:

- A single defector will benefit from not following the protocol
- Once sufficiently many nodes ignore the protocol, every participant loses

The example is a simple instance of the classical prisoner's dilemma, but game theory offers a much richer way of describing interactions among entities and coalitions

Game-Theoretical Models (2)

Game-theoretical models are also of interest where conventional adversary models may be too strong

- Conventionally, adversaries are assumed to always behave in a way that is optimal
- Adversaries may follow strategies that are not always optimal
- Similarly, adversary behaviour may depend on information that is itself incomplete

One of the key questions in (cyber) security is the optimal deployment of (finite) resources

- Conventionally, this is based on risk assessment — but adversaries can observe this deployment and adapt his strategy to avoid security controls
- Observations will not be perfect, making the use of Stackelberg competitions an attractive model

Further material in the unit on Attack Modelling. . .

Summary

This section provided material on

- Graph-based models and applications
- Agent-based models
- Game-theoretical modelling